ALMA MATER STUDIORUM
UNIVERSITÀ DI BOLOGNA

UNIVERSITY OF BOLOGNA

DEPARTMENT OF STATISTICAL SCIENCES

Master degree in Quantitative Finance

Conditional GAN

Supervisors:

Pietro Rossi

Giovanni Della Lunga

Candidate:

Matteo Rofrano

ACADEMIC YEAR 2024/2025

Contents

1 Background **3**
1.1 CGAN formulation 4
1.2 Possible improviment to CGAN 5

Chapter 1

Background

for each trajectory $j = 1, ..., J$ we define the SDE of the log-price X_t^j as:

$$\begin{aligned} dX_t^j &= -\frac{1}{2}\sigma_j^2 dt + \sigma_j dW_t \\ \text{with } X_0^j &= \log(S_0^j) \end{aligned} \tag{1.1}$$

Where S_0^j is the raw price at time 0 of trajectory j, σ_j is the volatility of trajectory j and W_t is a standard Brownian motion.
x_0^j and σ_j are randomly sampled for each trajectory such that $x_0^j \in [0, 10]$ and $\sigma_j \in (0.0, 1.0]$.

In this context our condition label y_j for a general data observation j is given by a vector containing the trajectory $y_j = [X_0^j, X_{dt}^j, X_{2dt}^j, ..., X_{Ndt}^j]$. So our condition data matrix Y has shape $J \times N$ where J is the number of trajectories (data points) and N is the number of time steps in each trajectory.

The SDE is discretized over a uniform time space $Ndt = t$ obtaining the following form adopted for the simulation of the trajectories:

$$X_{t+dt}^j = X_0^j - \frac{1}{2}\sigma_j^2 dt + \sigma_j \sqrt{dt} Z_t \tag{1.2}$$

Where $Z_t \sim \mathcal{N}(0, 1)$.
The procedure to build the condition data matrix Y is summarized in Algorithm 1.

Algorithm 1 Algorithm for trajectory simulation and condition data matrix Y creation

```
Define the number of simulation J.
Initialize an empty data matrix Y of shape J × N.
Set iteration counter j ← 0.
repeat
    Randomly sample X_0^j ∈ [0.0, 10.0] and σ_j ∈ (0.0, 1.0].
    Evolve X_t^j based on the discretized SDE for 0, dt, ..., Ndt steps.
    Store the trajectory [X_0^j, X_dt^j, X_2dt^j, ..., X_Ndt^j] in the j-th row of Y.
    Increment counter j ← j + 1.
until j = J
return Y
```

In $t = Ndt$ we have the following information: the final log-price X^j_{Ndt} of each trajectory j and the corresponding volatility σ_j. The goal of the cGAN is to learn the distribution of the next time interval $Ndt + \Delta t$ which is a normal distribution $\mathcal{N} \sim (X^j_{Ndt} - \frac{1}{2}\sigma_j^2 \Delta t, \sigma_j \Delta t)$. So the data matrix X, that in classical implementations of GANs are images, is in this case a vector containing the parametrization of the distribution. In this case a vector of two elements: the mean and the standard deviation.

1.1 CGAN formulation

GANs are often formalized as a minimax game, where the generator (G) seeks to minimize the value function V while the discriminator (D) aims to maximize it:

$$\min_G \max_D V(D, G) = \mathbb{E}_{\boldsymbol{x} \sim p_{\text{data}}(\boldsymbol{x})}[\log D(\boldsymbol{x})] + \mathbb{E}_{z \sim p_z(z)}[\log(1 - D(G(z)))] \tag{1.3}$$

The discriminator outputs a value $D(x)$, which represents the probability that x is the real vector containing mean and variance of the normal distribution. Its goal is to correctly classify the real parameters as real and generated parameters as fake, effectively maximizing the likelihood of the observed data. To measure this, we use cross-entropy loss, as is common in deep learning, expressed as $plog(1 - p)$. For real parameters, the true label p equals 1, while for generated parameters the label is 0. With this setup, the discriminator's objective becomes:

$$\max_D V(D) = \mathbb{E}_{\boldsymbol{x} \sim p_{\text{data}}(\boldsymbol{x})}[\log D(\boldsymbol{x})] + \mathbb{E}_{z \sim p_z(z)}[\log(1 - D(G(z)))] \tag{1.4}$$

where $\mathbb{E}_{\boldsymbol{x} \sim p_{\text{data}}(\boldsymbol{x})}[\log D(\boldsymbol{x})]$ define how good the discriminator recognize real parameters. and $\mathbb{E}_{z \sim p_z(z)}[\log(1 - D(G(z)))]$ define how good the discriminator recognizes generated parameters.
The generator's objective is to produce distribution parameters that maximize $D(x)$, pushing the discriminator to classify them as real:

$$\min_G V(G) = \mathbb{E}_{z \sim p_z(z)}[\log(1 - D(G(z)))] \tag{1.5}$$

Where $\mathbb{E}_{z \sim p_z(z)}[\log(1 - D(G(z)))]$ define how good the generator is in fooling the discriminator.

In the Conditional GAN (CGAN), the generator learns to generate a fake sample with a specific condition, in our case the past trajectory, rather than a generic sample from unknown noise distribution. The CGAN objective function is given by:

$$\min_G \max_D V(D, G) = \mathbb{E}_{\boldsymbol{x} \sim p_{\text{data}}(\boldsymbol{x})}[\log D(\boldsymbol{x} \mid \boldsymbol{y})] + \mathbb{E}_{z \sim p_z(z)}[\log(1 - D(G(z \mid \boldsymbol{y})))] \tag{1.6}$$

Academic findings highlighted in (*1*) suggest that one of the principal failure modes of GANs arises when the generator converges to a parameter configuration that yields only a limited variety of outputs. Existing research indicates that the occurrence of mode collapse is attributed to the establishment of a local equilibrium within the minimax optimization framework. A further critical challenge involves the vanishing gradient problem, which emerges when the discriminator becomes overly proficient, causing the generator's gradients to approach zero and consequently impeding its learning process.

1.2 Possible improviment to CGAN

Different strategies have been adopted to improve training stability and convergence of GANs.

- **Wasserstein GAN (WGAN)**: WGAN (2) introduces the Wasserstein distance (Earth Mover's Distance) as a new loss function for GANs. This distance metric provides a more meaningful measure of the difference between the real and generated data distributions, leading to improved training stability and convergence. The WGAN objective function is given by $\min_G \max_{D \in \mathcal{D}} E_{\boldsymbol{x} \sim p_d}[D(\boldsymbol{x})] - E_{\widehat{\boldsymbol{x}} \sim p_g}[D(\widehat{\boldsymbol{x}})]$.

- **DRAGAN**: this framework was introduced to address these challenges by incorporating a gradient penalty directly into the GAN architecture. The authors in (3) observed that the local equilibria frequently manifest steep discriminator gradients in the vicinity of certain real data samples. They further demonstrated that such degenerate local equilibria can be mitigated by applying a gradient penalty to the objective function, the method adopted is called DRAGAN.

- A tip suggested in (1) to do not use batch normalization after adding the gradient penalty in order to improve model performances.

Bibliography

(*1*) Rao Fu et al. “Time series simulation by conditional generative adversarial net”. In: *arXiv preprint arXiv:1904.11419* (2019).

(*2*) Martin Arjovsky, Soumith Chintala, and Léon Bottou. “Wasserstein generative adversarial networks”. In: *International conference on machine learning*. PMLR. 2017, pp. 214–223.

(*3*) Naveen Kodali et al. “On convergence and stability of gans”. In: *arXiv preprint arXiv:1705.07215* (2017).